İTÜ Matematik Bölümü Ders Yönetim Sisteminin Modern Web Teknolojileri ile Yeniden Tasarımı

Recep İlcan

090210303

İstanbul Teknik Üniversitesi
Fen Edebiyat Fakültesi
Matematik Mühendisliği Bölümü

MAT 4901
Matematik Müh. Tasarımı I
2025-2026 Güz Dönemi

Ocak 2026

Özet

Bu çalışmada, İstanbul Teknik Üniversitesi Matematik Bölümü'nün mevcut ders yönetim sisteminin modern web teknolojileri kullanılarak yeniden tasarlanması ele alınmaktadır. Eski sistem PHP, HTML ve Bootstrap gibi teknolojiler üzerine kurulu monolitik bir yapıya sahipken, yeni sistem React 19, TypeScript, Vite ve TailwindCSS v4 gibi güncel teknolojiler kullanılarak component tabanlı bir Single Page Application (SPA) olarak geliştirilmiştir. Bu modernizasyon çalışması ile kullanıcı deneyimi önemli ölçüde iyileştirilmiş, sayfa navigasyonu mantıksal bir yapıya kavuşturulmuş ve sistem bakım kolaylığı açısından sürdürülebilir hale getirilmiştir.

İçindekiler

1 Giriş **4**
- 1.1 Proje Bağlantıları . . . 4

2 Mevcut Sistemin Analizi ve Problemler **4**
- 2.1 Mimari Problemler . . . 4
- 2.2 Hedef Kitle Yönetimi Problemi . . . 4
- 2.3 İçerik Organizasyonu Problemleri . . . 5

3 Çözüm Yaklaşımı ve Teknoloji Seçimi **5**
- 3.1 Teknoloji Yığını . . . 5
- 3.2 shadcn/ui Kullanımı . . . 5

4 Uygulanan Çözümler **6**
- 4.1 Birleşik Hedef Kitle Yönetimi . . . 6
  - 4.1.1 Teknik Implementasyon . . . 6
  - 4.1.2 Kullanıcı Deneyimi İyileştirmeleri . . . 7
- 4.2 Modüler Sayfa Yapısı . . . 7
- 4.3 Duyuru Sistemi Yeniden Tasarımı . . . 7
  - 4.3.1 Özellikler . . . 8
  - 4.3.2 Custom Hook Kullanımı . . . 8
- 4.4 Ders Detay Sayfası Tasarımı . . . 8
  - 4.4.1 Tab Tabanlı Navigasyon . . . 8
  - 4.4.2 URL Senkronizasyonu . . . 9
  - 4.4.3 Materyal Yönetimi . . . 9
- 4.5 Yardım Sayfası (/help) . . . 9
  - 4.5.1 İçerik Yapısı . . . 9

5 Teknik Mimari **10**
- 5.1 Proje Klasör Yapısı . . . 10
- 5.2 Mock Data Yapısı ve Servis Katmanı . . . 10
  - 5.2.1 Mock Veri Klasör Yapısı . . . 11
  - 5.2.2 Veri Modelleri . . . 11
  - 5.2.3 Servis Katmanı Tasarımı . . . 11
  - 5.2.4 Mock Veri Özellikleri . . . 12
  - 5.2.5 API Entegrasyonu İçin Hazırlık . . . 12
- 5.3 State Management Stratejisi . . . 13
- 5.4 Routing Yapısı . . . 13

6 Performans Optimizasyonları **14**
- 6.1 Code Splitting ve Lazy Loading . . . 14
- 6.2 Query Caching . . . 14
- 6.3 Skeleton Loading States . . . 14

7 Çoklu Dil Desteği **14**

8 Tema Desteği **15**

9 Sonuç ve Değerlendirme **15**

10 Gelecek Çalışmalar **15**

11 Kaynakça **16**

1 Giriş

İstanbul Teknik Üniversitesi Matematik Bölümü, öğrencilerine ders materyalleri, duyurular, sınav bilgileri ve çeşitli akademik kaynakları sunmak amacıyla bir web sitesi işletmektedir. Mevcut sistem 2016 yılından bu yana `https://mathavuz.itu.edu.tr/` adresinde hizmet vermekte olup, PHP/HTML/Bootstrap teknolojileri üzerine inşa edilmiştir. Zamanla artan kullanıcı beklentileri, mobil cihaz kullanımının yaygınlaşması ve modern web standartlarının gelişmesi ile birlikte sistemin yeniden tasarlanması ihtiyacı ortaya çıkmıştır.

Bu bitirme projesi kapsamında, mevcut sistemin tüm işlevselliği korunarak modern web teknolojileri ile yeniden yazılması hedeflenmiştir. Proje, kullanıcı deneyimini ön planda tutarak, erişilebilirlik, performans ve bakım kolaylığı ilkelerine uygun şekilde geliştirilmiştir.

1.1 Proje Bağlantıları

Açıklama	**URL**
Eski Sistem (Mevcut)	`https://mathavuz.itu.edu.tr/`
Yeni Sistem (Bu Proje)	`https://web.itu.edu.tr/ilcan21/`

2 Mevcut Sistemin Analizi ve Problemler

2.1 Mimari Problemler

Eski sistem monolitik bir yapıya sahip olup, tüm içerik tek bir sayfa üzerinde sunulmaktaydı. Bu durum aşağıdaki problemlere yol açmaktaydı:

- **Sayfa Yüklenme Süresi:** Tüm içeriğin tek sayfada bulunması, sayfa yüklenme süresini olumsuz etkilemekteydi.
- **Navigasyon Karmaşıklığı:** Kullanıcılar aradıkları bilgiye ulaşmak için uzun sayfa kaydırma işlemleri yapmak zorundaydı.
- **URL Yapısı:** Anlamlı URL'ler bulunmadığından, belirli içeriklere doğrudan bağlantı vermek mümkün değildi.
- **SEO Uyumsuzluğu:** Tek sayfa yapısı arama motoru optimizasyonunu olumsuz etkilemekteydi.

2.2 Hedef Kitle Yönetimi Problemi

Eski sistemde iki farklı hedef kitle için iki ayrı site bulunmaktaydı:

1. **MAT Havuz Dersleri Sitesi:** MAT 103/E ve MAT 104/E gibi tüm mühendislik fakültesi öğrencilerine açık dersler için
2. **MAT Bölümü Sitesi:** Matematik bölümü öğrencilerine özel dersler için

Bu ayrım ciddi kullanılabilirlik sorunlarına neden olmaktaydı:

- **Giriş Zorunluluğu:** Kullanıcı giriş yapmadan iki site arasında geçiş yapamamaktaydı. Havuz sitesinden bölüm sitesine erişmek için önce kullanıcı girişi gerekmekteydi.
- **Tutarsız Arayüz:** İki site arasında tasarım tutarsızlıkları mevcuttu. Örneğin, havuz sitesinin hero bölümünde bölüm sitesine geçiş bağlantısı bulunurken, bölüm sitesinde bu imkân yoktu.
- **Çift Bakım Yükü:** İki ayrı sitenin bakımı, güncelleme süreçlerini karmaşıklaştırmaktaydı.

2.3 İçerik Organizasyonu Problemleri

Eski sistemde içerikler kategorize edilmeden tek sayfada listeleniyordu:

- Duyurular, ders materyalleri, sınav arşivi, çalışma soruları, önemli bilgiler ve yardım dokümanları hepsi aynı sayfada yer alıyordu.
- Belirli bir derse ait materyallere ulaşmak için tüm sayfanın taranması gerekiyordu.
- Arama ve filtreleme özellikleri bulunmuyordu.

3 Çözüm Yaklaşımı ve Teknoloji Seçimi

3.1 Teknoloji Yığını

Yeni sistem için modern ve sürdürülebilir bir teknoloji yığını seçilmiştir:

Tablo 1: Kullanılan Teknolojiler ve Versiyonları

Kategori	**Teknoloji**	**Versiyon**
Framework	React	19.2.0
Dil	TypeScript	5.9.3
Build Tool	Vite	7.2.4
Styling	TailwindCSS	4.1.18
UI Components	shadcn/ui + Radix UI	En güncel
Routing	React Router	7.12.0
State Management	TanStack Query	5.90.19
Internationalization	i18next	25.7.4
Animations	Framer Motion	12.26.2

3.2 shadcn/ui Kullanımı

Projede UI component kütüphanesi olarak **shadcn/ui** tercih edilmiştir. Bu seçimin temel gerekçeleri:

- **Erişilebilirlik (Accessibility):** shadcn/ui, Radix UI primitives üzerine inşa edilmiştir. Radix UI, WCAG standartlarına uygun, klavye navigasyonu ve ekran okuyucu desteği sunan erişilebilir componentler sağlamaktadır.

- **Özelleştirilebilirlik:** Componentler doğrudan proje içine kopyalanarak kullanıldığından, tam kontrol sağlanmaktadır. Bu yaklaşım, kütüphane güncellemelerinden bağımsızlık ve ihtiyaca özel modifikasyon imkânı sunmaktadır.
- **TailwindCSS Entegrasyonu:** shadcn/ui, TailwindCSS ile native olarak çalışacak şekilde tasarlanmıştır. Bu sayede tutarlı bir styling deneyimi elde edilmektedir.
- **Hazır Componentler:** Button, Card, Input, Dropdown Menu, Tabs, Accordion, Navigation Menu gibi yaygın kullanılan componentler production-ready kalitede sunulmaktadır.

Projede kullanılan başlıca shadcn/ui componentleri:

```
components/ui/
|-- accordion.tsx         # Yardim sayfasi SSS bolumu
|-- alert.tsx             # Bilgilendirme mesajlari
|-- badge.tsx             # Etiketler ve durumlar
|-- button.tsx            # Tum butonlar
|-- card.tsx              # Ders ve duyuru kartlari
|-- dropdown-menu.tsx     # Filtre menuleri
|-- input.tsx             # Arama kutulari
|-- navigation-menu.tsx   # Ana navigasyon
|-- tabs.tsx              # Ders detay sekmeleri
|-- tooltip.tsx           # Ipucu bilgileri
`-- sheet.tsx             # Mobil menu
```

4 Uygulanan Çözümler

4.1 Birleşik Hedef Kitle Yönetimi

Eski sistemdeki iki ayrı site problemi, tek bir uygulama içinde **Audience Provider** pattern'ı ile çözülmüştür.

4.1.1 Teknik Implementasyon

React Context API kullanılarak global bir audience state yönetimi oluşturulmuştur:

```
// providers/audience-provider.tsx
export function AudienceProvider({
  children,
  defaultAudience = 'department',
  storageKey = 'app-audience'
}) {
  const [audience, setAudienceState] = useState<AudienceKey>(
    () => localStorage.getItem(storageKey) || defaultAudience
  );

  useEffect(() => {
    localStorage.setItem(storageKey, audience);
  }, [audience, storageKey]);

  return (
```

```
    <AudienceContext.Provider value={{ audience, setAudience }}>
      {children}
    </AudienceContext.Provider>
  );
}
```

4.1.2 Kullanıcı Deneyimi İyileştirmeleri

- **Header'da Geçiş Butonu:** Kullanıcı girişi gerektirmeksizin, header bölümüne eklenen "Audience Toggle" butonu ile iki hedef kitle arasında anlık geçiş sağlanmaktadır.
- **LocalStorage Persistance:** Kullanıcının tercihi tarayıcıda saklanarak, sonraki ziyaretlerde otomatik olarak hatırlanmaktadır.
- **Hero Bölümü Sadeleştirmesi:** Eski sistemdeki tutarsız hero bölümü geçiş bağlantıları kaldırılmış, bunun yerine header'daki buton üzerinden bilgilendirici tooltip ile yönlendirme yapılmaktadır.
- **Dinamik İçerik Filtreleme:** Seçilen hedef kitleye göre dersler, duyurular ve materyaller otomatik olarak filtrelenmektedir.

4.2 Modüler Sayfa Yapısı

Eski sistemdeki tek sayfa yapısı, mantıksal olarak gruplandırılmış ayrı sayfalara bölünmüştür:

Tablo 2: Sayfa Yapısı Karşılaştırması

Eski Sistem	**Yeni Sistem**
Tüm dersler ana sayfada liste	/courses - Arama ve filtreleme destekli ders kataloğu
Duyurular ana sayfada	/announcements - Ayrı sayfa, gelişmiş filtreleme
Ders materyalleri ana sayfada	/courses/:id/materials - Derse özel materyal sayfası
Sınav arşivi ana sayfada	/courses/:id/materials - Materyal tipi filtresi ile
Önemli bilgiler ana sayfada	/help - Ayrı yardım sayfası

4.3 Duyuru Sistemi Yeniden Tasarımı

Eski sistemde duyurular basit bir liste halinde ana sayfada gösterilmekteydi. Yeni sistemde kapsamlı bir duyuru yönetimi oluşturulmuştur:

4.3.1 Özellikler

- **Ayrı Sayfa (/announcements):** Duyurular artık kendi sayfasında, pagination ile listelenmektedir.
- **Gerçek Zamanlı Arama:** Kullanıcılar duyuru başlığı veya içeriğinde arama yapabilmektedir.
- **Gelişmiş Filtreleme:**
  - Yeni/okunmamış duyuruları gösterme
  - Derse göre filtreleme (ders seçici dropdown)
  - Tarih aralığına göre filtreleme
- **Detay Sayfası (/announcements/:id):** Her duyurunun kendi detay sayfası bulunmaktadır.
- **Derse Özel Erişim:** Ders detay sayfasındaki "Duyurular" sekmesinden o derse ait duyurulara doğrudan erişim sağlanmaktadır.

4.3.2 Custom Hook Kullanımı

Duyuru verilerinin yönetimi için özel bir hook geliştirilmiştir:

```
const {
    announcements,
    total,
    isLoading,
    searchQuery,
    showOnlyNew,
    selectedCourses,
    dateFilter,
    activeFilterCount,
    updateSearch,
    toggleCourse,
    goToNextPage,
    clearAllFilters,
} = useAnnouncements({ audience, initialLimit: 5 });
```

4.4 Ders Detay Sayfası Tasarımı

Yeni sistemin en önemli iyileştirmelerinden biri, her ders için özel detay sayfalarının oluşturulmasıdır.

4.4.1 Tab Tabanlı Navigasyon

Ders detay sayfası (/courses/:courseId) dört ana sekmeden oluşmaktadır:

1. **Genel Bakış (Overview):** Ders hakkında özet bilgiler
2. **Materyaller (Materials):** Ders notları, ödevler, sınavlar

3. **Duyurular (Announcements):** Derse özel duyurular
4. **Ders Bilgileri (Info):** Detaylı ders bilgileri

Not: Mevcut prototipte "Genel Bakış" ve "Ders Bilgileri" sekmeleri henüz içerik ile doldurulmamıştır. Bu sayfalar, backend API entegrasyonu sırasında gerçek ders verileri ile tamamlanacaktır.

4.4.2 URL Senkronizasyonu

Tab değişimleri URL'e yansıtılmaktadır. Bu sayede:

- Kullanıcılar belirli sekmelere doğrudan bağlantı paylaşabilmektedir
- Tarayıcı geri/ileri butonları beklendiği gibi çalışmaktadır
- Sayfa yenilendiğinde kullanıcı aynı sekmede kalmaktadır

4.4.3 Materyal Yönetimi

Ders materyalleri sayfası (/courses/:courseId/materials) aşağıdaki özelliklere sahiptir:

- **Tip Filtreleme:** Ders notu, ödev, sınav, doküman, video, bağlantı kategorileri
- **Sıralama Seçenekleri:** En yeni, en eski, başlığa göre
- **Arama:** Materyal başlığında gerçek zamanlı arama
- **Pagination:** Server-side sayfalama ile performans optimizasyonu

4.5 Yardım Sayfası (/help)

Eski sistemde ana sayfanın "Önemli Bilgiler" bölümünde yer alan içerikler, ayrı bir yardım sayfasına taşınmıştır.

4.5.1 İçerik Yapısı

- **Hızlı Erişim Kartları:**
  - Kısa Sınav Rehberi (PDF)
  - Optik Yanıt Kağıdı (PDF)
  - Mazeret Sınavları Bilgilendirmesi (PDF)
  - Sınav Sonuçlarına İtiraz (PDF)
- **Accordion Yapısında Detaylı Bilgiler:**
  - Sınav kuralları ve düzenlemeleri
  - Not hesaplama yöntemleri
  - İletişim bilgileri

5 Teknik Mimari

5.1 Proje Klasör Yapısı

```
src/
|-- components/
|   |-- common/            # Paylasilan componentler
|   |   |-- course-card.tsx
|   |   |-- announcement-card.tsx
|   |   |-- material-card.tsx
|   |   |-- page-header.tsx
|   |   `-- app-header/
|   `-- ui/                # shadcn/ui componentleri
|-- hooks/                 # Custom React hooks
|   |-- use-courses.ts
|   |-- use-materials.ts
|   |-- use-announcements.ts
|   `-- use-document-title.ts
|-- services/              # API servis katmani
|   |-- courses.service.ts
|   |-- materials.service.ts
|   `-- announcements.service.ts
|-- providers/             # Context providers
|   |-- theme-provider.tsx
|   |-- language-provider.tsx
|   `-- audience-provider.tsx
|-- locale/                # i18n ceviri dosyalari
|   |-- tr/
|   `-- en/
|-- types/                 # TypeScript type definitions
|-- mock/                  # Mock veri dosyalari
|   |-- courses.ts
|   |-- materials.ts
|   `-- announcements.ts
|-- pages/                 # Sayfa componentleri
|   |-- public/
|   |   |-- home/
|   |   |-- courses/
|   |   |-- announcements/
|   |   `-- help/
|   |-- protected/
|   `-- errors/
`-- routes/                # Routing konfigurasyonu
```

5.2 Mock Data Yapısı ve Servis Katmanı

Bu projede gerçek bir backend API kullanılmamıştır. Bunun yerine, frontend geliştirme sürecini hızlandırmak ve uygulamanın işlevselliğini test etmek amacıyla **mock (sahte) veri** yapısı oluşturulmuştur. Servis methodları, gelecekte gerçek API entegrasyonuna geçişi kolaylaştıracak şekilde tasarlanmıştır.

5.2.1 Mock Veri Klasör Yapısı

```
src/mock/
|-- courses.ts        # Ders verileri (13 ders)
|-- materials.ts      # Ders materyalleri (500+ materyal)
`-- announcements.ts  # Duyurular (12 duyuru)
```

5.2.2 Veri Modelleri

Her veri tipi için TypeScript interface'leri tanımlanmıştır:

```
// types/course.ts
interface Course {
    id: string;
    code: string;
    title: { tr: string; en: string };
    students: number;
    color: string;
    audience: AudienceKey; // "department" | "common"
}

// types/course-material.ts
interface CourseMaterial {
    id: string;
    courseId: string;
    title: { tr: string; en: string };
    type: MaterialType; // "lecture"|"assignment"|"exam"|...
    date: string;
    size?: string;
    url?: string;
    description?: { tr: string; en: string };
}

// types/announcement.ts
interface Announcement {
    id: string;
    title: { tr: string; en: string };
    description: { tr: string; en: string };
    date: string;
    courseId: string;
    audience: AudienceKey;
    isNew: boolean;
}
```

5.2.3 Servis Katmanı Tasarımı

Servis fonksiyonları, gerçek bir API'den veri çeker gibi tasarlanmıştır. Bu sayede ileride backend entegrasyonu yapılırken yalnızca servis dosyalarının güncellenmesi yeterli olacaktır:

```
// services/courses.service.ts
export const getCourses = async (params: GetCoursesParams):
```

```
    Promise<GetCoursesResponse> => {
    const { audience, offset, limit, search, sortBy } = params;

    // Simulate API delay
    await new Promise(resolve => setTimeout(resolve, 100));

    let courses = MockCourses;

    // Filter by audience
    if (audience) {
        courses = courses.filter(c => c.audience === audience);
    }

    // Search in code and title
    if (search?.trim()) {
        const q = search.toLowerCase();
        courses = courses.filter(c =>
            c.code.toLowerCase().includes(q) ||
            c.title.tr.toLowerCase().includes(q) ||
            c.title.en.toLowerCase().includes(q)
        );
    }

    // Pagination
    const total = courses.length;
    const paginatedCourses = courses.slice(offset, offset + limit);

    return { data: paginatedCourses, total, hasMore: offset+limit < total };
};
```

5.2.4 Mock Veri Özellikleri

- **Gerçekçi Veri:** Mock veriler, İTÜ Matematik Bölümü'nün gerçek ders kodları ve içerikleri baz alınarak oluşturulmuştur (MAT 103/E, MAT 104/E, MAT 251/E vb.).
- **Çift Dil Desteği:** Tüm metin alanları hem Türkçe hem İngilizce olarak tanımlanmıştır.
- **Hedef Kitle Ayrımı:** Her veri kaydı `audience` alanı ile "department" veya "common" olarak etiketlenmiştir.
- **Simüle Edilmiş Gecikme:** API çağrıları 100ms yapay gecikme ile simüle edilmekte, bu sayede loading state'leri test edilebilmektedir.
- **Server-Side İşlemler:** Filtreleme, arama, sıralama ve sayfalama işlemleri servis katmanında yapılmakta, gerçek bir backend davranışı taklit edilmektedir.

5.2.5 API Entegrasyonu İçin Hazırlık

Mevcut mimari, backend entegrasyonunu kolaylaştıracak şekilde tasarlanmıştır:

1. **Servis Soyutlaması:** Componentler doğrudan mock veriye erişmez, servis fonksiyonları üzerinden veri alır.

2. **Type Safety:** TypeScript interface'leri sayesinde API response yapısı önceden tanımlıdır.

3. **TanStack Query Entegrasyonu:** Veri çekme işlemleri TanStack Query ile yapılmakta, cache ve refetch mekanizmaları hazırdır.

4. **Tek Değişiklik Noktası:** Gerçek API'ye geçişte yalnızca servis dosyalarındaki `MockData` import'ları `fetch` çağrıları ile değiştirilecektir.

5.3 State Management Stratejisi

Projede üç katmanlı bir state yönetimi stratejisi uygulanmıştır:

1. **Server State (TanStack Query):** API'den gelen veriler için. Caching, background refetch, pagination desteği sağlar.

2. **Global Client State (Context API):** Tema, dil ve hedef kitle gibi uygulama genelinde paylaşılan durumlar için.

3. **Local Component State (useState):** Form inputları, açık/kapalı durumları gibi component-specific durumlar için.

5.4 Routing Yapısı

React Router v7 ile nested routing implementasyonu:

```
const router = createBrowserRouter([
  {
    path: "/:lang?",
    element: <MainLayout />,
    children: [
      { index: true, element: <HomePage /> },
      {
        path: "courses",
        children: [
          { index: true, element: <CoursesPage /> },
          {
            path: ":courseId",
            element: <CoursePage />,
            children: [
              { index: true, element: <CourseOverviewPage /> },
              { path: "materials", element: <CourseMaterialsPage /> },
              { path: "announcements", element: <CourseAnnouncementsPage /> },
              { path: "info", element: <CourseInfoPage /> }
            ]
          }
        ]
      },
      { path: "announcements", element: <AnnouncementsPage /> },
```

```
      { path: "help", element: <HelpPage /> },
    ]
  }
]);
```

6 Performans Optimizasyonları

6.1 Code Splitting ve Lazy Loading

Tüm sayfa componentleri `React.lazy()` ile dinamik olarak import edilmektedir:

```
const CoursesPage = lazy(() => import("@/pages/public/courses"));
const HelpPage = lazy(() => import("@/pages/public/help"));
```

Bu yaklaşım ile initial bundle size minimize edilmekte ve sayfalar yalnızca ihtiyaç duyulduğunda yüklenmektedir.

Bu sayede uygulamanın ilk yüklenme süresi azaltılmış ve kullanıcı deneyimi iyileştirilmiştir. Sayfalar yalnızca ihtiyaç duyulduğunda yüklendiği için gereksiz JavaScript kodlarının indirilmesi engellenmiş, uygulama daha verimli ve ölçeklenebilir bir yapıya kavuşmuştur.

6.2 Query Caching

TanStack Query ile server state caching:

```
const { data, isLoading } = useQuery({
    queryKey: ["courses", queryParams],
    queryFn: () => getCourses(queryParams),
    placeholderData: keepPreviousData,
    staleTime: 5 * 60 * 1000, // 5 dakika cache
});
```

TanStack Query, sunucudan alınan verilerin geçici olarak saklanmasını sağlayarak aynı verinin tekrar tekrar istenmesini engeller. Bu sayede uygulama daha hızlı çalışır ve kullanıcı sayfalar arasında geçiş yaparken daha akıcı bir deneyim yaşar.

6.3 Skeleton Loading States

Kullanıcı deneyimini iyileştirmek için veri yüklenirken skeleton componentler gösterilmektedir. Bu yaklaşım, içerik yüklenme süresinin algılanan uzunluğunu azaltmaktadır.

7 Çoklu Dil Desteği

Sistem, Türkçe ve İngilizce olmak üzere iki dili desteklemektedir. i18next kütüphanesi ile namespace-based çeviri yapısı kullanılmıştır:

```
locale/
|-- tr/
|   |-- common.json        # Genel ceviriler
|   |-- courses.json       # Ders sayfasi cevirileri
|   |-- announcements.json
```

```
|   |-- help.json
|   `-- errors.json
`-- en/
    `-- (ayni yapi)
```

8 Tema Desteği

Sistem üç tema modunu desteklemektedir:

- **Light Mode:** Açık tema
- **Dark Mode:** Koyu tema (varsayılan)
- **System:** İşletim sistemi tercihine göre otomatik

Tema tercihi LocalStorage'da saklanarak kalıcı hale getirilmektedir.

9 Sonuç ve Değerlendirme

Bu bitirme projesi kapsamında, İTÜ Matematik Bölümü ders yönetim sistemi modern web teknolojileri kullanılarak baştan sona yeniden tasarlanmıştır. Geliştirilen yeni sistem `https://web.itu.edu.tr/ilcan21/` adresinde canlı olarak erişime sunulmuştur.

Gerçekleştirilen başlıca iyileştirmeler:

1. **Birleşik Platform:** İki ayrı site tek bir uygulamada birleştirilmiş, giriş gerektirmeksizin hedef kitle geçişi sağlanmıştır.
2. **Modüler Yapı:** Tek sayfa yapısından, mantıksal olarak gruplandırılmış sayfa yapısına geçilmiştir.
3. **Gelişmiş Arama ve Filtreleme:** Tüm sayfalara gerçek zamanlı arama ve çoklu filtreleme özellikleri eklenmiştir.
4. **Ders Odaklı Navigasyon:** Her ders için özel detay sayfaları oluşturulmuş, materyaller ve duyurular derse göre organize edilmiştir.
5. **Modern UX:** Dark mode, responsive tasarım, animasyonlar ve skeleton loading ile kullanıcı deneyimi iyileştirilmiştir.
6. **Çoklu Dil Desteği:** Türkçe ve İngilizce dil seçenekleri eklenmiştir.
7. **Bakım Kolaylığı:** Component-based mimari ve TypeScript ile kod kalitesi artırılmıştır.

10 Gelecek Çalışmalar

- Backend API entegrasyonu (mock data'dan gerçek API'ye geçiş)
- **Ders Özeti ve Bilgileri Sayfaları:** Ders detay sayfasındaki "Genel Bakış" ve "Ders Bilgileri" sekmelerinin gerçek verilerle doldurulması (ders açıklaması, ön koşullar, öğretim üyeleri, AKTS bilgileri, haftalık ders planı vb.)

- Kullanıcı kimlik doğrulama sistemi
- Kişisel not takibi ve yoklama modülleri

11 Kaynakça

Kaynaklar

[1] React - A JavaScript library for building user interfaces. `https://react.dev` (Erişim Tarihi: Ocak 2026)

[2] TypeScript - JavaScript with syntax for types. `https://www.typescriptlang.org` (Erişim Tarihi: Ocak 2026)

[3] Vite - Next Generation Frontend Tooling. `https://vite.dev` (Erişim Tarihi: Ocak 2026)

[4] Tailwind CSS - A utility-first CSS framework. `https://tailwindcss.com` (Erişim Tarihi: Ocak 2026)

[5] shadcn/ui - Beautifully designed components built with Radix UI and Tailwind CSS. `https://ui.shadcn.com` (Erişim Tarihi: Ocak 2026)

[6] Radix UI - Unstyled, accessible components for building high-quality design systems. `https://radix-ui.com` (Erişim Tarihi: Ocak 2026)

[7] React Router - Declarative routing for React. `https://reactrouter.com` (Erişim Tarihi: Ocak 2026)

[8] TanStack Query - Powerful asynchronous state management for TS/JS, React, Solid, Vue and Svelte. `https://tanstack.com` (Erişim Tarihi: Ocak 2026)

[9] i18next - internationalization framework. `https://i18next.com` (Erişim Tarihi: Ocak 2026)

[10] Motion - Production-ready animation library for React. `https://motion.dev` (Erişim Tarihi: Ocak 2026)

[11] Lucide - Beautiful & consistent icon toolkit. `https://lucide.dev` (Erişim Tarihi: Ocak 2026)

[12] Magic UI - React components to build beautiful landing pages. `https://magicui.design` (Erişim Tarihi: Ocak 2026)

[13] Aceternity UI - Modern UI components for React. `https://aceternity.com` (Erişim Tarihi: Ocak 2026)